Structural Verification Lab: A Reproducible Integration of Numerical Research Modules

21 October 2026

Abstract

Can a shared material and geometry definition expose inconsistent error levels between static beam compliance and elastic buckling? This software-methods paper consolidates 6 existing research implementations into a self-contained laboratory and adds a connected numerical experiment. Source snapshots, licenses, tests and original papers are retained with commit-level provenance and file hashes. The contribution is explicit interface verification and reproducible orchestration, rather than a new numerical method or a claim of physical validation. The reported experiment distinguishes shared-model consistency from numerical error and documents where component assumptions prevent further automatic coupling.

1 Research scope and architecture

Beam discretization, stability, nonlinear equilibrium, plasticity, adaptive meshes and boundary enforcement. The included implementations are shearscope, bucklemap, foldtrace, yieldledger, meshbeacon, boundarybalance [1, 2, 3, 4, 5, 6]. Each remains a separately installable Python package; native C++ kernels retain their package-specific interfaces. The root runner builds the modules, executes tests in separate processes, checks imported file hashes and produces the new integration report. This process avoids test-module name collisions and makes failed checks visible instead of treating a directory collection as validated integration.

2 Governing model and connecting assumptions

For a uniform cantilever of length L, bending rigidity EI and shear rigidity $S = \kappa GA$, the small-displacement tip compliance under a unit force is

$$C_T = \frac{L^3}{3EI} + \frac{L}{S}.$$

The Euler–Bernoulli cantilever critical load is $P_E = \pi^2 EI/(4L^2)$. These different beam theories share a bending-only scalar identity,

$$\frac{P_E}{L}\left(C_T - \frac{L}{S}\right) = \frac{\pi^2}{12}.$$

Subtracting the shear term is essential: the identity does not equate the complete stiffness matrices or establish a shear-deformable buckling theory. ShearScope supplies full- and reduced-quadrature Timoshenko compliance; BuckleMap supplies Hermite finite-element Euler loads.

3 Reproducible experiment

The original implementations are reused through public Python APIs, including their separately compiled C++ element kernels. The new adapter creates a one-metre cantilever with width 0.02 m, height 0.01 m, Young's modulus 70 GPa, Poisson ratio 0.3 and shear correction 5/6. Both solvers receive the same length and bending rigidity. Four meshes (4, 8, 16 and 32 elements) are compared. Static error uses the independent closed-form Timoshenko compliance; load error uses the Euler cantilever formula. The product check measures cross-model parameter consistency. Run `python suite.py experiment` after installing the suite to regenerate the machine-readable results. All table entries below are generated from that report; no numerical result is copied from a component paper. Component source commits and per-file hashes are stored in the suite manifest. The experiment and integration test are versioned alongside this manuscript.

4 Measured integration results

Elements	Load rel. error	Reduced rel. error	Full rel. error	Product
4	3.277e-05	1.562e-02	9.950e-01	0.809643
8	2.060e-06	3.906e-03	9.804e-01	0.819256
16	1.290e-07	9.765e-04	9.260e-01	0.821664
32	7.919e-09	2.441e-04	7.579e-01	0.822266

Reference product: 0.822467033.

The recorded study shows much slower improvement of compliance than of the Hermite buckling load. Full shear quadrature remains severely locked on the slender-beam meshes, even though the buckling result is accurate. Agreement of one output therefore does not certify another solver's approximation. Reduced quadrature recovers the expected bending-compliance limit in this example. FoldTrace, YieldLedger, MeshBeacon and BoundaryBalance retain separate nonlinear, constitutive, adaptive and boundary-verification studies under the shared runner; their assumptions are not inserted into this elastic identity.

5 Verification and limitations

The root test command runs each component's Python tests and the shared-model contract tests, recording return codes and complete output. Independent tests check modulus and geometric scaling, and refinement toward the analytical beam limits at slenderness ratios 20 and 100. C++ extensions are compiled during installation. A separate `native` command builds and executes the four C++17 invariant tests through CTest. Cross-platform CI runs both test commands; sanitizer, distribution and full historical reproduction campaigns are outside this gate. The paper build rejects unresolved references, overfull boxes and manuscripts over five pages. Imported bytes use explicit file hashes; a documented line-ending repair and one attributes-file patch make the provenance gate portable without changing numerical source.

The common experiment is linear, slender and uniform, with ideal clamping. It does not connect plasticity to buckling, trace post-buckling branches, establish nonlinear stability, or validate a physical beam. Refinement evidence applies to the chosen meshes and material scaling. The imported modules support additional independent experiments, but their historical results are not new integration measurements.

6 Conclusion

The integrated repository supplies a common execution and evidence interface while preserving the assumptions that make each numerical component interpretable. The connected experiment makes a specific cross-component contract testable. Further coupling should be treated as new research work, with independently derived reference cases and a revised paper, rather than inferred from successful package installation.

References

[1] shearscope. Versioned implementation and numerical study. https://github.com/racostacme-come/shearscope. Exact imported commit recorded in `suite.json`.

[2] bucklemap. Versioned implementation and numerical study. https://github.com/racostacme-come/bucklemap. Exact imported commit recorded in `suite.json`.

[3] foldtrace. Versioned implementation and numerical study. https://github.com/racostacme-come/foldtrace. Exact imported commit recorded in `suite.json`.

[4] yieldledger. Versioned implementation and numerical study. https://github.com/racostacme-come/yieldledger. Exact imported commit recorded in `suite.json`.

[5] meshbeacon. Versioned implementation and numerical study. https://github.com/racostacme-come/meshbeacon. Exact imported commit recorded in `suite.json`.

[6] boundarybalance. Versioned implementation and numerical study. https://github.com/racostacme-come/boundarybalance. Exact imported commit recorded in `suite.json`.